ASSIGNEMENT 22.3

V.Sruthi

2303A51308

Batch-05

TASK – 01

Prompt :

1. Generate an python program to shortlist candidates for a software engineer position. The goal is to select the best candidate based on qualifications, skills, and experience. Describe the steps, criteria, and weighting of attributes
2. Generate an algorithm to shortlist candidates for a software engineer position. The goal is to ensure diversity and inclusion while considering skills, experience, and qualifications. Describe the steps, criteria, and weighting of attributes.

Code :

```
'''#Generate an python program to shortlist candidates for a software engineer
position. The goal is to select the best candidate based on qualifications,
skills, and experience. Describe the steps, criteria, and weighting of
attributes
# candidate_shortlisting.py
# Step 1: Define the criteria and their weights
criteria = {

    "qualifications": 0.4,  # Weight for educational qualifications
    "skills": 0.4,          # Weight for technical skills
    "experience": 0.2       # Weight for work experience
}

# Step 2: Create a list of candidates with their attributes
candidates = [
    {"name": "Alice", "qualifications": 8, "skills": 9, "experience": 5},
    {"name": "Bob", "qualifications": 7, "skills": 8, "experience": 6},
    {"name": "Charlie", "qualifications": 9, "skills": 7, "experience": 4}
]

# Step 3: Calculate the score for each candidate based on the criteria and
weights
def calculate_score(candidate):
```

```
    score = (candidate["qualifications"] * criteria["qualifications"] +
             candidate["skills"] * criteria["skills"] +
             candidate["experience"] * criteria["experience"])
    return score
# Step 4: Shortlist candidates based on their scores
shortlisted_candidates = sorted(candidates, key=calculate_score, reverse=True)
# Step 5: Display the shortlisted candidates
print("Shortlisted Candidates:")
for candidate in shortlisted_candidates:
    print(f"{candidate['name']} - Score: {calculate_score(candidate):.2f}")'''
# Short explanation
#The program defines three criteria for shortlisting candidates:
qualifications, skills, and experience, each with a specific weight. It then
creates a list of candidates with their respective attributes. The
`calculate_score` function computes a weighted score for each candidate based
on the defined criteria. Finally, the candidates are sorted by their scores in
descending order, and the shortlisted candidates along with their scores are
displayed. This approach allows for an objective evaluation of candidates
based on multiple attributes.
#Generate an algorithm to shortlist candidates for a software engineer
position. The goal is to ensure diversity and inclusion while considering
skills, experience, and qualifications. Describe the steps, criteria, and
weighting of attributes.
# candidate_shortlisting_diversity.py
# Step 1: Define the criteria and their weights
criteria = {
    "qualifications": 0.3,  # Weight for educational qualifications
    "skills": 0.3,          # Weight for technical skills
    "experience": 0.2,      # Weight for work experience
    "diversity": 0.2        # Weight for diversity and inclusion factors
}
# Step 2: Create a list of candidates with their attributes
candidates = [
    {"name": "Alice", "qualifications": 8, "skills": 9, "experience": 5,
"diversity": 1},
    {"name": "Bob", "qualifications": 7, "skills": 8, "experience": 6,
"diversity": 0},
    {"name": "Charlie", "qualifications": 9, "skills": 7, "experience": 4,
"diversity": 1}
]
# Step 3: Calculate the score for each candidate based on the criteria and
weights
def calculate_score(candidate):
    score = (candidate["qualifications"] * criteria["qualifications"] +
             candidate["skills"] * criteria["skills"] +
             candidate["experience"] * criteria["experience"] +
             candidate["diversity"] * criteria["diversity"])
    return score
```

```python
# Step 4: Shortlist candidates based on their scores
shortlisted_candidates = sorted(candidates, key=calculate_score, reverse=True)
# Step 5: Display the shortlisted candidates
print("Shortlisted Candidates:")
for candidate in shortlisted_candidates:
    print(f"{candidate['name']} - Score: {calculate_score(candidate):.2f}")
# Short explanation
#The algorithm incorporates diversity and inclusion as a criterion alongside
qualifications, skills, and experience. Each criterion is assigned a specific
weight to reflect its importance in the shortlisting process. Candidates are
evaluated based on their attributes, and a composite score is calculated for
each candidate. The candidates are then sorted by their scores, ensuring that
those who contribute to diversity and inclusion are given due consideration
while still valuing their qualifications and skills.
```

Output :

```
PROBLEMS 53    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTM
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/U
ing/22.3.py"
Shortlisted Candidates:
Alice - Score: 7.80
Bob - Score: 7.20
Charlie - Score: 7.20
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/U
ing/22.3.py"
Shortlisted Candidates:
Alice - Score: 6.30
Charlie - Score: 5.80
Bob - Score: 5.70
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

Comparing the two algorithms, the first focuses solely on qualifications, skills, and experience, while the second algorithm adds an additional layer of consideration for diversity and inclusion. This ensures a more holistic approach to candidate selection, promoting a more inclusive workplace while still maintaining a focus on the essential attributes required for the software engineer position.

TASK – 02

Prompt : Generate a program to process student academic records, including roll number, marks, and email. The program should store, retrieve, and display the data efficiently. Additionally, include best practices for handling sensitive data securely, such as masking personal information, encrypting data, and ensuring privacy protection.

Code :

```
#Generate a program to process student academic records, including roll
number, marks, and email. The program should store, retrieve, and display the
data efficiently. Additionally, include best practices for handling sensitive
data securely, such as masking personal information, encrypting data, and
ensuring privacy protection.
# student_records.py
import hashlib
import re
class StudentRecord:
    def __init__(self, roll_number, marks, email):
        self.roll_number = roll_number
        self.marks = marks
        self.email = self.encrypt_email(email)

    def encrypt_email(self, email):
        # Simple encryption using SHA-256 (for demonstration purposes)
        return hashlib.sha256(email.encode()).hexdigest()

    def mask_email(self):
        # Mask the email for display (e.g., ****@example.com)
        masked = re.sub(r'([^@]+)(@.*)', r'\1****\2', self.email)
        return masked
class StudentRecordsManager:
    def __init__(self):
        self.records = {}

    def add_record(self, roll_number, marks, email):
        if roll_number in self.records:
            print("Record already exists for this roll number.")
            return
        self.records[roll_number] = StudentRecord(roll_number, marks, email)

    def retrieve_record(self, roll_number):
        record = self.records.get(roll_number)
        if record:
            return {
                "roll_number": record.roll_number,
                "marks": record.marks,
```

```
                "email": record.mask_email()
            }
        else:
            print("Record not found.")
            return None

    def display_all_records(self):
        for roll_number, record in self.records.items():
            print(f"Roll Number: {record.roll_number}, Marks: {record.marks},
Email: {record.mask_email()}")
# Example usage
manager = StudentRecordsManager()
manager.add_record("101", 85, "john.doe@example.com")
manager.add_record("102", 90, "jane.smith@example.com")
print(manager.retrieve_record("101"))
manager.display_all_records()
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devis/AppData/Local/Python/pythoncore-3.14-64/pyt
ing/22.3.py"
{'roll_number': '101', 'marks': 85, 'email': '836f82db99121b3481011f16b49dfa5fbc714a0d1b1b9f784a1ebbbf5b39577f'}
Roll Number: 101, Marks: 85, Email: 836f82db99121b3481011f16b49dfa5fbc714a0d1b1b9f784a1ebbbf5b39577f
Roll Number: 102, Marks: 90, Email: f2d1f1c853fd1f4be1eb5060eaae93066c877d069473795e31db5e70c4880859
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation : The program defines a `StudentRecord` class to represent individual student records, which includes methods for encrypting and masking email addresses to enhance privacy. The `StudentRecordsManager` class manages the collection of student records, allowing for adding new records, retrieving specific records, and displaying all records. The email addresses are encrypted using SHA-256 for secure storage, and when displayed, they are masked to protect personal information. This approach ensures that sensitive data is handled securely while still allowing for efficient data management and retrieval.

TASK – 03

Prompt : Generate a secure payment processing script for an online shopping system. Ensure proper handling of sensitive data, input validation, and protection against common security risks**.**

Code :

```
#Generate a secure payment processing script for an online shopping system.
Ensure proper handling of sensitive data, input validation, and protection
against common security risks.
# payment_processing.py
import re
import hashlib
def validate_card_number(card_number):
    # Simple regex for validating card number (Visa, MasterCard, Amex)
    pattern = r'^(?:4[0-9]{12}(?:[0-9]{3})?|5[1-5][0-9]{14}|3[47][0-9]{13})$'
    return re.match(pattern, card_number) is not None
def validate_expiry_date(expiry_date):
    # Validate expiry date in MM/YY format
    pattern = r'^(0[1-9]|1[0-2])\/?([0-9]{2})$'
    return re.match(pattern, expiry_date) is not None
def validate_cvv(cvv):
    # Validate CVV (3 or 4 digits)
    pattern = r'^\d{3,4}$'
    return re.match(pattern, cvv) is not None
def hash_card_data(card_number, expiry_date, cvv):
    # Hash card data for secure storage
    data = f"{card_number}|{expiry_date}|{cvv}"
    return hashlib.sha256(data.encode()).hexdigest()
def process_payment(card_number, expiry_date, cvv):
    if not validate_card_number(card_number):
        print("Invalid card number!")
        return False
    if not validate_expiry_date(expiry_date):
        print("Invalid expiry date!")
        return False
    if not validate_cvv(cvv):
        print("Invalid CVV!")
        return False

    # Hash card data before processing
    hashed_data = hash_card_data(card_number, expiry_date, cvv)
    print(f"Processing payment with hashed data: {hashed_data}")

    # Simulate payment processing (replace with actual API call)
    print("Payment processed successfully!")
    return True
# Example usage
card_number = input("Enter card number: ")
expiry_date = input("Enter expiry date (MM/YY): ")
cvv = input("Enter CVV: ")
process_payment(card_number, expiry_date, cvv)
```

Output :

```
ing/22.3.py
Enter card number: 1234 1234 5678 2546
Enter expiry date (MM/YY): 12/30
Enter CVV: 365
Invalid card number!
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

This payment processing script includes functions to validate credit card information such as the card number, expiry date, and CVV using regular expressions. It also hashes the card data using SHA-256 before processing the payment to enhance security. The script simulates payment processing and provides feedback on the success or failure of the transaction while ensuring that sensitive data is handled securely.

TASK – 04

Prompt : Generate a basic program for an autonomous traffic signal control system that adjusts signal timings based on traffic density. Include clear logic, safety checks, and error handling. Also add comments explaining each part of the code.

Code :

```
#Generate a basic program for an autonomous traffic signal control system that
adjusts signal timings based on traffic density. Include clear logic, safety
checks, and error handling. Also add comments explaining each part of the
code.
# traffic_signal_control.py
import time
# Simulated traffic density data (in vehicles per minute)
traffic_density = {
    "North-South": 30,
    "East-West": 25
}
# Function to calculate signal timings based on traffic density
def calculate_signal_timings(density):
    # Base time for green signal (in seconds)
    base_time = 30
    # Adjust green signal time based on traffic density
    if density > 40:
        return base_time + 20  # Increase green time for high traffic
    elif density > 20:
        return base_time + 10  # Moderate increase for medium traffic
    else:
```

```
        return base_time  # Default time for low traffic
# Function to control traffic signals
def control_traffic_signals():
    try:
        while True:
            # Get current traffic density for both directions
            ns_density = traffic_density["North-South"]
            ew_density = traffic_density["East-West"]
            print(f"Current Traffic Density - North-South: {ns_density}
vehicles/min, East-West: {ew_density} vehicles/min")

            # Calculate signal timings for both directions
            ns_green_time = calculate_signal_timings(ns_density)
            ew_green_time = calculate_signal_timings(ew_density)

            # Simulate traffic signal control
            print(f"North-South Green Time: {ns_green_time} seconds")
            print(f"East-West Green Time: {ew_green_time} seconds")

            # Simulate the passage of time for the signals
            time.sleep(5)  # Wait for 5 seconds before the next update

    except KeyboardInterrupt:
        print("Traffic signal control system stopped.")
    except Exception as e:
        print(f"An error occurred: {e}")
# Start the traffic signal control system
if __name__== "__main__":
    control_traffic_signals()
```

Output :

```
ing/22.3.py"
East-West Green Time: 40 seconds
Current Traffic Density - North-South: 30 vehicles/min, East-West: 25 vehicles/min
North-South Green Time: 40 seconds
East-West Green Time: 40 seconds
Current Traffic Density - North-South: 30 vehicles/min, East-West: 25 vehicles/min
North-South Green Time: 40 seconds
East-West Green Time: 40 seconds
Current Traffic Density - North-South: 30 vehicles/min, East-West: 25 vehicles/min
North-South Green Time: 40 seconds
East-West Green Time: 40 seconds
Traffic signal control system stopped.
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devis/AppData/Local/
ing/22.3.py"
```

Explanation : The program simulates an autonomous traffic signal control system that adjusts signal timings based on traffic density. It defines a function

to calculate green signal timings based on the current traffic density for two directions (North-South and East-West). The main function continuously monitors traffic density, calculates the appropriate signal timings, and simulates the control of traffic signals. The program includes error handling to manage unexpected issues and allows for graceful shutdown using a keyboard interrupt.

TASK – 05

Prompt : Generate a program for an aircraft maintenance fault detection system. Include logic for detecting faults, handling errors, and ensuring reliability. Also mention the limitations of AI-generated code and the need for human validation in safety-critical systems.

Code :

```
#Generate a program for an aircraft maintenance fault detection system.
Include logic for detecting faults, handling errors, and ensuring reliability.
Also mention the limitations of AI-generated code and the need for human
validation in safety-critical systems.
# aircraft_maintenance.py
class AircraftMaintenanceSystem:
    def __init__(self):
        self.faults = []

    def detect_fault(self, sensor_data):
        # Simulated fault detection logic
        if sensor_data.get("engine_temp") > 100:
            self.faults.append("Engine Overheating")
        if sensor_data.get("oil_pressure") < 20:
            self.faults.append("Low Oil Pressure")
        if sensor_data.get("hydraulic_pressure") < 30:
            self.faults.append("Low Hydraulic Pressure")

    def handle_errors(self):
        if not self.faults:
            print("No faults detected. Aircraft is in good condition.")
        else:
            print("Faults detected:")
            for fault in self.faults:
                print(f"- {fault}")
            # Simulated error handling logic
            print("Initiating maintenance protocols...")
# Example usage
if __name__== "__main__":
```

```
    maintenance_system = AircraftMaintenanceSystem()
    sensor_data = {
        "engine_temp": 105,
        "oil_pressure": 15,
        "hydraulic_pressure": 25
    }
    maintenance_system.detect_fault(sensor_data)
    maintenance_system.handle_errors()
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> &
3.py"
Faults detected:
- Engine Overheating
- Low Oil Pressure
- Low Hydraulic Pressure
Initiating maintenance protocols...
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

Limitations of AI-generated code:

1. AI-generated code may not cover all edge cases or scenarios, especially in complex systems like aircraft maintenance.

2. AI may not fully understand the safety implications of certain faults or errors, leading to incomplete or incorrect handling logic.

3. Human validation is crucial in safety-critical systems to ensure that the code is reliable and meets all necessary safety standards. AI can assist in generating code, but it cannot replace the expertise and judgment of human engineers in ensuring the safety and reliability of aircraft maintenance systems.

In conclusion, while AI can be a valuable tool in generating code for fault detection systems, it is essential to have human oversight and validation to ensure the safety and reliability of such systems, especially in critical applications like aircraft maintenance.

TASK – 06

Prompt : Generate a sample program (any language) with proper comments and include a section explaining how to ethically use AI-generated code, including citation, verification, and avoiding misuse.

Code :

```
#Generate a sample program (any language) with proper comments and include a
section explaining how to ethically use AI-generated code, including citation,
verification, and avoiding misuse.
# ethical_ai_code.py
# Sample Python program to demonstrate ethical use of AI-generated code
def calculate_area(radius):
    """Calculate the area of a circle given its radius."""
    import math
    return math.pi * (radius ** 2)
# Example usage
if __name___== "__main__":
    r = 5
    area = calculate_area(r)
    print(f"The area of a circle with radius {r} is: {area:.2f}")
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C
3.py"
The area of a circle with radius 5 is: 78.54
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

When using AI-generated code, it is crucial to adhere to ethical guidelines to ensure responsible and fair use. Here are some key principles to follow:

1. Citation: Always give credit to the source of the AI-generated code. If you are using code generated by an AI model, mention that it was created with the assistance of AI and specify the model or tool used.

2. Verification: Before using AI-generated code in production or sharing it with others, thoroughly review

and test the code to ensure it functions as intended. AI-generated code may contain errors or unintended consequences, so it is essential to verify its correctness and safety.

3. Avoiding Misuse: Do not use AI-generated code for malicious purposes, such as creating harmful software, spreading misinformation, or violating privacy. Always consider the ethical implications of the code you are using and strive to use it in a way that benefits society.

By following these guidelines, you can ensure that you are using AI-generated code responsibly and ethically, contributing to a positive and constructive technological environment.